Vietnamese-German University

Electrical and Computer Engineering Study Program

Title:

IoT-SecureGate: Unified Software Platform for Secure Edge IoT Gateways

Monitoring Application and Secure Edge Gateway

By

Name	Student ID
Nguyen Tran Son Tung	10222035
Nguyen Minh Bach	10222002
Pham Huynh Tri Thuc	10222058

First Supervisor: Assoc. Prof. Dr. Chan Thai Truyen Dai

Second Supervisor: Mr. Tran Quang Nhu

BACHELOR THESIS PROJECT

An integrated account of the IoT-SecureGate platform.

Vietnamese-German University, 2026

Abstract

In manufacturing and other factory-floor IoT deployments, edge sensors continuously send measurements into internal systems that drive monitoring, control, and production decisions. Those edge devices sit outside the trusted core of the plant network, so unauthorized clients, stolen credentials, or malicious publishers can inject false data or abuse messaging channels. Without a controlled entry point, the plant cannot reliably decide which devices may speak, what they may publish, or when a source should be cut off.

IoT-SecureGate is a security gateway for this setting. It sits between edge devices and the internal network and gives administrators a single place to govern incoming sensor messages, monitor the gateway and data path through operational metrics, and detect denied or invalid traffic so misuse can be stopped. The platform authenticates and authorizes devices before they may publish, protects inbound transmission, keeps the health of that path visible, and supports operator response when traffic is denied or abnormal.

This thesis presents the design and implementation of that secure gateway. Relative to earlier research prototypes, the work also delivers the gateway as a ready-to-use software package for operators. The report evaluates the gateway functions in operation and notes directions in which the system can be improved in future work.

Keywords: IoT security, edge gateway, MQTT, device authentication, manufacturing IoT, gateway monitoring.

Acknowledgements

We would like to thank our supervisors for their guidance, feedback, and support throughout this project. We also thank the academic staff who provided advice during the development and preparation of this thesis.

Contents

Acknowledgements i

Abbreviations vii

1 Introduction 1

1.1 Background and Motivation . . . 1
1.2 Problem Statement . . . 1
1.3 Project Aim and Objectives . . . 2
1.4 Report Organization . . . 3

2 Technical Background and Related Work 4

2.1 MQTT and Edge Messaging . . . 4
2.2 Transport Security and Mutual TLS . . . 4
2.2.1 Confidentiality and integrity on the wire . . . 4
2.2.2 Mutual authentication . . . 5
2.2.3 Operational limits of transport security . . . 5
2.3 Application-Layer Message Protection . . . 5
2.4 Metrics, Logs, and Operational Visibility . . . 6
2.5 Related Work . . . 6
2.5.1 Secure MQTT at the broker . . . 6
2.5.2 Edge monitoring with metrics and logs . . . 7
2.5.3 Position of this work . . . 7

3 System Requirements 8

3.1 Operational Context . . . 8
3.2 Functional Requirements . . . 9
3.3 Non-Functional Requirements . . . 11
3.4 Scope . . . 12

3.4.1 In scope . . . 12
3.4.2 Out of scope . . . 13
3.5 Threat Boundaries . . . 13

4 General System Architecture **15**
4.1 System Top-Level Architecture . . . 15
4.2 Monitoring Application Overview . . . 16
4.3 Gateway Overview . . . 17
4.4 Control Plane and Data Plane . . . 18
4.5 End-to-End Inbound Path . . . 18
4.6 Layered Message Protection Overview . . . 19
4.7 Deployment Topology and Ports . . . 21
4.8 Trust Material and Secrets . . . 22
4.9 Relation to Following Chapters . . . 22

5 Monitoring Application **23**
5.1 Architecture of the Monitoring Application . . . 23
5.2 Interaction with the Gateway . . . 23
5.2.1 Gateway discovery and installation . . . 23
5.2.2 Sensor and fake-sensor onboarding . . . 23
5.3 Functions and Operational Workflows . . . 23
5.3.1 Local monitoring preparation . . . 23
5.3.2 Observing metrics and transmission activity . . . 23
5.3.3 Control and laboratory exercise . . . 23
5.3.4 Lifecycle of the operator installation . . . 23

6 Gateway **24**
6.1 Architecture of the Gateway . . . 24
6.2 Agent Operation and MQTT Handling . . . 24
6.3 Inbound Processing Path . . . 24
6.4 Gateway APIs and Control Interfaces . . . 24
6.5 Detection and Response Behaviour . . . 24

7 Encryption and Decryption **25**
7.1 Protection Model . . . 25
7.2 Transport Security and Mutual TLS . . . 25

7.3 Application-Layer Envelope 25
7.4 Encryption and Decryption Workflow 25
7.5 Schema Validation and Accepted Storage 25

8 Evaluation **26**
8.1 Evaluation Goals and Method 26
8.2 Admission and Authentication 26
8.3 Protected Transmission and Storage 26
8.4 Monitoring and Detection 26
8.5 Resource Use and Limitations 26

9 Conclusion and Future Work **27**
9.1 Summary 27
9.2 Contributions 27
9.3 Future Work 27

List of Figures

4.1 Top-level architecture of IoT-SecureGate: monitoring application, gateway, and sensors. 16
4.2 Overview of the monitoring application on the operator PC. 17
4.3 Overview of the gateway runtime and its links to operator observability. . . 18
4.4 Control plane (SSH) and data plane (MQTTS and HTTP observability) between operator PC, gateway, and sensors. 18
4.5 End-to-end inbound path from sensor publish to storage and operator observability. 19
4.6 Layered protection on the inbound path: MQTTS transport and application envelope v1. 20
4.7 Deployment topology and principal ports on a trusted LAN. 21
4.8 Overview of trust material on the operator PC and on the gateway. 22

List of Tables

1.1 Organization of the report. 3

3.1 Functional requirements for IoT-SecureGate. 9
3.2 Non-functional requirements for IoT-SecureGate. 11

4.1 Protection layers on the gateway inbound path (overview). 19
4.2 Operator PC ports for the monitoring application. 21
4.3 Gateway device ports. 21

Abbreviations

Term	**Meaning**
ACL	Access Control List
API	Application Programming Interface
CA	Certificate Authority
CPU	Central Processing Unit
CSR	Certificate Signing Request
DMEC	Daily Message Encryption Certificate
GUI	Graphical User Interface
IDS	Intrusion Detection System
IoT	Internet of Things
JSON	JavaScript Object Notation
MQTT	Message Queuing Telemetry Transport
mTLS	Mutual Transport Layer Security
PKI	Public Key Infrastructure
RAM	Random Access Memory
SSH	Secure Shell
TLS	Transport Layer Security
UI	User Interface

Chapter 1

Introduction

1.1 Background and Motivation

Modern manufacturing and factory environments rely on Internet of Things (IoT) edge devices—sensors on machines, lines, and cells—to stream measurements into internal monitoring and control systems. That inbound telemetry supports production visibility, quality checks, and operational decisions. The communication pattern is typically publish/subscribe over MQTT: many constrained clients send data through a broker toward applications inside the plant [1].

The edge is also an exposed surface. Devices may be physically accessible, credentials may leak, and a compromised or spoofed publisher can inject false readings or abuse messaging channels that reach systems behind the plant boundary. As factories connect more sensors, the risk is no longer limited to a single host: an untrusted edge client that can publish freely becomes a path into the internal data path that operators and receivers depend on.

A security gateway is therefore required at the boundary between edge devices and the internal network. Its job is to decide which devices may transmit, under which identity and topic policy, with which cryptographic protections, and to keep that inbound path observable so that misuse can be detected and stopped. IoT-SecureGate is developed for that role: a secure edge gateway that authenticates devices, controls transmission into the plant side, protects message content, and supports operator response when traffic is denied or invalid.

1.2 Problem Statement

Without a dedicated gateway, factory IoT traffic often arrives through a broker that accepts clients with weak or manually maintained credentials, little binding between device identity and allowed topics, and limited visibility into failed or abusive publishes. Operators then face three concrete failures:

- **Uncontrolled admission.** Unknown or poorly identified devices can reach the broker and publish toward internal consumers.
- **Unprotected or weakly protected transmission.** Payloads may travel without

mutual authentication, without application-layer confidentiality after TLS termination, or without checks that the message shape matches what the gateway expects.

- **Weak response to misuse.** Denied connections and invalid messages may leave only raw broker logs, with no clear path to alert the operator, revoke publish rights, or quarantine a hostile source.

The central problem of this thesis is how to operate a secure gateway end to end for such environments: admit only registered edge devices, enforce authentication and authorization on the path into the internal network, protect and validate messages, observe gateway and message activity, and restrict devices that violate policy.

1.3 Project Aim and Objectives

The aim is to design, implement, and evaluate IoT-SecureGate as one secure gateway system that governs edge device access and inbound data transmission for factory and similar IoT deployments. The thesis describes that system as a whole: the operator monitoring application and the gateway that admits, protects, observes, and responds on the inbound sensor path.

The concrete objectives follow that function:

- **Objective 1: Secure device admission.** Ensure that only registered devices with valid credentials and certificates can connect and publish on allowed topics.
- **Objective 2: Controlled and protected transmission.** Carry inbound sensor data over MQTTS, decrypt and validate envelopes on the gateway, and forward accepted data onto the internal storage and messaging path.
- **Objective 3: Monitor gateway metrics and transmission activity.** Observe the health and metrics of the gateway and the live behaviour of inbound transmission flows for analysis.
- **Objective 4: Alert and automatically restrict invalid sensors.** Detect denied and invalid messages, alert the administrator when such traffic appears, and provide mechanisms that can automatically block invalid sensors, with manual controls when the operator needs to intervene.

Building on earlier Senior Project prototypes, this work also delivers the gateway as a ready-to-use software package (desktop operator application and gateway agent) so that the security functions above can be deployed and operated without reconstructing the system from separate research scripts.

1.4 Report Organization

Ch.	Title	Content
2	Technical Background and Related Work	Messaging, transport security, message protection, monitoring building blocks, and related research.
3	System Requirements	Operational context, functional and non-functional requirements, scope, and threat boundaries.
4	General System Architecture	System overview diagrams: two runtimes, planes, inbound path, protection layers, ports, and trust material.
5	Monitoring Application	Operator runtime architecture, interaction with the gateway, and operational workflows.
6	Gateway	Edge runtime architecture, agent and MQTT handling, gateway APIs, and detection and response.
7	Encryption and Decryption	Transport and application-layer message protection workflows on the inbound path.
8	Evaluation	Assessment of the system in operation.
9	Conclusion and Future Work	Summary of the work and directions for later improvement.

Table 1.1: Organization of the report.

Chapter 2

Technical Background and Related Work

This chapter introduces the technical ideas needed to read the system design that follows. It covers edge messaging with MQTT, transport authentication with mutual TLS, application-layer message protection, and operational metrics and logs. A short related-work section places the project next to common practice in secure MQTT brokers and edge monitoring. Implementation of IoT-SecureGate begins in later chapters.

2.1 MQTT and Edge Messaging

MQTT is a lightweight publish/subscribe messaging protocol widely used for IoT telemetry [1]. Clients do not send data directly to every consumer. They connect to a *broker*, which accepts *publish* operations and delivers messages to clients that have *subscribed* to matching *topics*.

A topic is a hierarchical name. Authorization at the broker decides which clients may publish or subscribe on which topics. That decision is separate from the network path the packets take: a client that can open a connection is not automatically allowed to use every topic.

In a factory or plant setting, many edge sensors publish measurements toward applications inside the internal network. The broker—often placed on a local gateway—becomes the controlled meeting point for that inbound traffic. Understanding MQTT therefore matters for any gateway that must admit devices, restrict topic use, and forward accepted data further inland.

2.2 Transport Security and Mutual TLS

2.2.1 Confidentiality and integrity on the wire

Without transport protection, MQTT credentials and payloads can be observed or altered on the network. TLS provides an encrypted and integrity-protected channel between client and server [3]. When MQTT runs over TLS (commonly called MQTTS), the broker presents a certificate so the client can verify that it is talking to the intended endpoint.

2.2.2 Mutual authentication

Server authentication alone does not prove who the client is. Mutual TLS (mTLS) additionally requires the client to present an X.509 certificate that the broker trusts [4]. The broker then binds the TLS identity to local policy before accepting the MQTT session.

Password authentication at the broker is a second, independent check. Mosquitto, for example, can require a username and password and can enforce topic access control lists (ACLs) that allow or deny publish and subscribe operations per client [2]. Valid passwords do not replace topic ACLs: a client may authenticate successfully and still be denied a forbidden topic.

The intended access decision can be stated as:

$$\text{AccessAllowed} = \text{Authenticated} \wedge \text{Authorized}\,. \tag{2.1}$$

Authentication establishes the identity claim (password and/or client certificate). Authorization checks the requested publish or subscribe against policy. Both must succeed for the operation to proceed.

2.2.3 Operational limits of transport security

TLS protects data while it travels between endpoints. After the broker terminates TLS, the application that reads the message sees plaintext unless a further protection layer is applied. Certificate lifecycle (issuance, renewal, trust store configuration) also belongs to operational security: a correct protocol choice still fails if devices use expired or untrusted certificates.

2.3 Application-Layer Message Protection

Application-layer protection addresses confidentiality and integrity of the payload *after* TLS termination, and can add checks that transport alone does not provide.

A typical pattern is an *envelope*: the sensor encrypts the measurement under a content key, wraps that key for the gateway, and sends one or more MQTT messages that the gateway can unwrap, decrypt, and validate. Authenticated encryption (for example AES-GCM) binds confidentiality to an integrity tag so that modified ciphertext is rejected.

Two further ideas matter for a gateway that must store or forward decoded measurements:

- **Schema validation.** After decrypt, the gateway checks that the JSON (or other) payload matches the shape expected for that device. A cryptographically valid message can still be invalid for the application.
- **Freshness / replay resistance.** Encryption does not by itself prove that a message is new. An application may reject repeated message identifiers or other freshness tokens so that a captured envelope cannot be replayed successfully.

Transport security and application-layer protection answer different questions. MQTTS

and mTLS answer who may connect and publish. The envelope answers whether the measurement remains confidential beyond the TLS hop and whether the decoded content is acceptable to the gateway.

After a message is accepted and decoded, a gateway typically retains it for local audit and forwards it onto an inland messaging or storage path so plant-side consumers can read the same measurements. The exact technologies used by IoT-SecureGate are described in the architecture chapters; the background point is that admission and decrypt are not the end of the inbound path.

2.4 Metrics, Logs, and Operational Visibility

A secure gateway that only enforces policy still leaves operators without evidence that the path is healthy. Operational visibility usually combines two kinds of signal:

- **Metrics** are numerical time series (CPU use, memory, message rates, processing delays). Prometheus is a common system for scraping metric endpoints and storing those series [5].
- **Logs** are event records (accepted messages, denied connections, decryption failures). Loki is a common system for aggregating and querying such logs [6].

Dashboards (for example Grafana) present metrics and logs so that administrators can inspect gateway health and transmission activity without reading raw service files. For an edge gateway, useful visibility includes both *host and service health* and *message-path activity*, including signs that traffic was denied or failed validation. Those signals are the conceptual basis for alerting and for deciding when a misbehaving source should be restricted.

2.5 Related Work

2.5.1 Secure MQTT at the broker

Production MQTT deployments commonly combine TLS, broker passwords, and topic ACLs so that only known clients can publish on allowed channels [1, 2]. Mutual TLS extends that model by also requiring a trusted client certificate, which is a standard pattern for strong device identity on the wire [3, 4]. Many gateways use password authentication and client certificates together with topic ACLs, so that transport identity and publish rights are both checked before a measurement is accepted.

That line of practice matches the security problems addressed later in this thesis: register a device, bind it to credentials and certificates, and enforce who may connect and publish at the broker. The background here is broker-side access control and message protection, not a machine-learning intrusion pipeline.

2.5.2 Edge monitoring with metrics and logs

On the operations side, scraping numerical metrics and collecting service logs is standard for keeping hosts and pipelines visible [5, 6]. Edge gateways use the same idea: expose health of the broker and processing path, and retain event records when a connection or message is rejected.

This thesis builds on that practice for gateway metrics and transmission activity. Denied connections and invalid messages appear as operational signals on that path—something the administrator can see and act on through alerts and publish restrictions.

2.5.3 Position of this work

IoT-SecureGate sits at the intersection of those two areas: a secure MQTT gateway with explicit device admission and message protection, plus operator visibility into gateway health and inbound transmission. The following chapters describe how that system is designed and implemented.

Chapter 3

System Requirements

This chapter states what IoT-SecureGate must achieve as a secure edge gateway. Requirements follow from the problem and objectives in Chapter 1 and from the operational setting of factory and similar IoT deployments. They describe the behaviour the system must provide; architecture and implementation appear in later chapters. Functional requirements are labelled FR-n; non-functional requirements are labelled NFR-n. Those identifiers are reused when the design and evaluation refer back to this chapter. The chapter also fixes the thesis scope and the security threat boundaries that apply to those requirements.

3.1 Operational Context

IoT-SecureGate sits between edge sensors and the internal plant network. Sensors on machines, lines, or cells publish measurements toward applications that operators and plant systems use for monitoring and control. The gateway is the controlled entry point on that inbound path: it admits devices, authenticates and authorizes publishes, protects and validates message content, stores accepted data for internal consumers, and keeps the path observable so that denied or invalid traffic can be detected and restricted.

The platform has two runtimes that operators treat as one system:

- **Monitoring application (operator PC).** A packaged desktop application with a local control service and a local observability stack (Loki, Prometheus, Grafana). From this application the administrator discovers and provisions gateways, registers devices, watches metrics and transmission activity, and applies day-to-day control actions.
- **Gateway (edge device).** An agent on a Linux edge host (for example a Raspberry Pi or small Ubuntu machine) together with an MQTTS broker and per-sensor messaging storage. Sensors connect here over MQTTS. The gateway decrypts and validates payloads, appends accepted data to a local log and to Kafka broker, ships logs to the operator stack, and exposes metrics for scraping.

The primary actor is the *administrator*: the person who installs the monitoring application, provisions gateways over the local network, registers sensors, and responds when Activities

show denied or invalid traffic. Secondary actors are *edge sensors* (real devices or laboratory fake publishers used for testing). Internal consumers that read stored Kafka topics or logs sit behind the gateway; they are not operators of the security controls themselves.

The intended deployment is a trusted plant or laboratory LAN. The operator PC and the gateway communicate for provision and day-2 actions over SSH. Sensor ingress uses MQTTS on the gateway. Metrics and logs return to the operator PC over HTTP on the same LAN. Routine operation is meant for users who can run an installer and use a graphical interface, without reconstructing the stack from separate research scripts.

3.2 Functional Requirements

Functional requirements cover admission, protected transmission, monitoring, detection and response, and the operator path that makes those functions deployable. They map to the four objectives in Chapter 1: secure device admission, controlled and protected transmission, gateway and activity monitoring, and alert plus restriction of invalid sensors.

Table 3.1: Functional requirements for IoT-SecureGate.

ID	Requirement
FR-1	The system shall admit only registered devices to the gateway broker. Registration shall bind a device identity to MQTT credentials, topic permissions, a client certificate under the gateway CA, and a reserved per-sensor storage topic.
FR-2	The gateway shall authenticate connecting clients with MQTT password verification and mutual TLS (client certificate required), and shall authorize publishes with topic ACLs so that a device may publish only on its allowed topics.
FR-3	Inbound sensor traffic shall use MQTTS on the gateway. After transport termination, the gateway shall decrypt application-layer envelopes (envelope v1), validate message shape against a registered schema, and reject traffic that fails those checks.
FR-4	Accepted message shall be written to a local sensor data log and produced to a per-device Kafka topic so that internal consumers and the operator observability path can use the data.
FR-5	The monitoring application shall discover and provision a gateway over SSH: install the agent, MQTTS broker configuration, and Kafka service; bootstrap gateway CA and server material; and record the gateway in a local registry with scrape and ship targets toward the operator stack.
FR-6	The administrator shall register and unregister devices from the monitoring application. Registration shall apply the admission material on the gateway (password, ACL, client certificate, crypto and schema files, Kafka topic) without requiring manual broker edits.

Table 3.1: Functional requirements for IoT-SecureGate (continued).

ID	Requirement
FR-7	The system shall expose gateway host and application health (agent, MQTT, decrypt, shipper, Kafka) as metrics scraped by the operator Prometheus instance, and shall present those metrics in the monitoring application.
FR-8	The system shall expose inbound transmission activity (message rates, delays, connected sensors, denied and invalid counts) through logs and summary APIs, and shall present that activity in the monitoring application.
FR-9	The gateway shall detect transport denials (failed authentication or ACL) and application-layer invalid messages (decrypt or schema failure), and shall make those events visible to the administrator as alerts in the monitoring application.
FR-10	The administrator shall be able to block and restore a device's publish rights (ACL change) from the monitoring application.
FR-11	The system shall support automatic restriction of a sensor source after a configurable count of invalid messages in a time window (firewall DROP of the source IP on the gateway), with an operator path to clear that quarantine.
FR-12	The system shall provide laboratory fake sensors on the gateway for testing admission, protected transmission, and misuse behaviours (including simulation of attack-like publish patterns) without requiring physical hardware for every test.
FR-13	The monitoring application shall install and run a local observability stack (Loki, Prometheus, Grafana) on the operator PC so that metrics and shipped logs are available without a separate monitoring deployment for routine use.
FR-14	Secrets used for gateway CA signing, day-2 privileged actions, and local admin functions shall be stored in the operator OS keyring (with a documented file fallback), and shall not be written into ordinary settings files or application logs.

FR-1 through FR-4 define the inbound security path on the gateway. FR-5, FR-6, FR-13, and FR-14 define the operator-side means to deploy and maintain that path. FR-7 and FR-8 cover visibility. FR-9 through FR-11 cover detection and response. FR-12 supports evaluation and laboratory exercise of the same path. A packaged machine-learning intrusion-detection module inside the installer is outside these requirements; deny and invalid-message handling, plus block and Auto prevent, are the protection controls in scope.

3.3 Non-Functional Requirements

Non-functional requirements constrain how the functional behaviour is delivered: who can operate the system, how it is packaged, how resources and capacity are bounded, and how trustworthy the operator path remains.

Table 3.2: Non-functional requirements for IoT-SecureGate.

ID	Requirement
NFR-1	**Usability.** Routine setup and day-to-day operation shall be possible through the packaged graphical monitoring application (install local monitoring, provision a gateway, register devices, view Metric and Activities, apply Control actions) without requiring the administrator to edit source code or maintain a multi-compose research stack for those tasks.
NFR-2	**Deployability.** The monitoring application shall be distributable as native packages for Windows and Linux so that the operator path can be installed on a PC without Docker or WSL as a default dependency.
NFR-3	**Gateway footprint.** The gateway runtime shall run as a service stack on a modest Linux edge host (agent, system Mosquitto, Kafka), suitable for small industrial or laboratory devices such as a Raspberry Pi–class machine.
NFR-4	**Capacity bound.** The number of registered sensors (real and laboratory fake) shall be limited by gateway disk budget for Kafka topic retention so that registration fails clearly when capacity is exhausted rather than filling the device unbounded.
NFR-5	**Data-path integrity.** Only messages that pass authentication, authorization, decryption, and schema validation shall be appended to the sensor data log and produced to Kafka; failed attempts shall contribute to deny / invalid signalling instead of silent acceptance.
NFR-6	**Confidentiality of operator secrets.** CA passphrases, gateway sudo credentials for day-2 playbooks, and the local admin PIN shall not appear in settings JSON or routine logs; provision of privileged commands shall pass passwords on SSH standard input rather than on remote process argument lists.
NFR-7	**Operational visibility.** Gateway metrics and shipped activity logs shall be available to the monitoring application through the local Prometheus and Loki instances so that the administrator can inspect health and invalid traffic during normal operation of the LAN deployment.
NFR-8	**Auditability of configuration change.** Device registration, publish block/allow, and related day-2 changes shall leave a durable record in the operator registry (and supporting audit state) so that which gateways and devices are under management remains inspectable after the action completes.

Table 3.2: Non-functional requirements for IoT-SecureGate (continued).

ID	Requirement
NFR-9	**Trusted-LAN assumption.** Control-plane SSH, Prometheus scrape, Loki push, and the Kafka external listener used by local Grafana are intended for a trusted plant or laboratory network; the system does not claim to be a public Internet-facing broker or multi-tenant cloud gateway.
NFR-10	**Automated delivery.** Changes in the shared codebase shall reach end-user installers through an automated continuous-integration pipeline: a push to the packaging branches builds the Windows executable and the Linux AppImage and publishes them to a release channel, so that distributing an updated monitoring application does not require a manual rebuild on a developer workstation.

Together, NFR-1 and NFR-2 express the shift from Senior Project prototypes to a usable packaged platform. NFR-3 and NFR-4 keep the edge host practical. NFR-5 through NFR-8 protect the quality of the security and operator path. NFR-9 states the deployment assumption that later architecture and evaluation sections rely on. NFR-10 requires that packaging and release of those installers follow an automated CI path from repository change to downloadable application.

3.4 Scope

This thesis covers IoT-SecureGate as one system: the monitoring application on the operator side and the gateway on the edge side. The functional and non-functional requirements above apply inside this scope.

3.4.1 In scope

- The monitoring application on the operator PC, including the desktop interface, control service, and local observability stack.
- Discovery and provision of the gateway, and day-to-day operation of registered gateways.
- Device registration and admission control (identity, MQTT credentials and topic permissions, mutual TLS client certificates).
- Protected inbound transmission on the gateway (MQTTS, application-layer encryption and decryption, schema validation, local log, and per-sensor Kafka topics).
- Monitoring of gateway metrics and of transmission activity on the inbound path.
- Detection of denied and invalid messages, operator alerts, manual block and allow of publish rights, and automatic restriction of invalid sensors.
- Laboratory fake sensors used to exercise admission, transmission, and misuse paths.

- The delivery strategy from source code to the end user, including packaging and automated distribution of the monitoring application for Windows and Linux.

3.4.2 Out of scope

- A full machine-learning intrusion-detection module inside the packaged application (only deny and invalid-message hooks are part of this system).
- Plant-wide systems beyond the gateway boundary, such as MES, SCADA, or cloud backends that consume stored sensor data.
- Sensor device firmware (for example ESP32 firmware trees).
- Full backup, restore, and remote-update product wizards beyond the existing remove and wipe flows.
- The earlier Streamlit-based control tooling from the Senior Project (replaced here by the desktop application and control service).

3.5 Threat Boundaries

Within the scope above, this section states the security perimeter and trust assumptions for the requirements. It does not repeat the product in/out lists; it states which threats the gateway is designed to address on the inbound edge path.

Security perimeter. IoT-SecureGate is responsible for the edge admission boundary: who may connect and publish, transport and application-layer protection of inbound messages, local recording of accepted data, visibility of denied and invalid traffic, and operator controls that block or quarantine misbehaving sources. FR-1 through FR-14 and NFR-1 through NFR-10 describe that perimeter in requirement form.

Assumed environment. The design assumes a plant or laboratory LAN in which the operator PC can reach the gateway for SSH and HTTP observability, and sensors can reach the gateway MQTTS port (NFR-9). Host firewall and network segmentation on that LAN remain under site administration. The Kafka external listener used by local Grafana is intended for that trusted LAN only.

Threats addressed at the edge. The requirements target misuse on the inbound sensor path: unknown or weakly identified publishers, failed authentication or authorization, unprotected or invalid payloads after TLS termination, and repeated invalid traffic that should alert the administrator and trigger restriction.

Threats outside this perimeter. Compromise of plant backends after data leaves gateway storage, physical destruction of hosts, supply-chain compromise of device firmware, and operation as a public Internet-facing or multi-tenant cloud broker remain outside the security claims of this thesis. Those exclusions align with the out-of-scope items in Section 3.4.

Chapters 4, 5, and 6 describe how the architecture meets FR-1–FR-14 and NFR-1–NFR-10 within these boundaries. Chapter 8 assesses the resulting system against the same requirements.

Chapter 4

General System Architecture

This chapter gives the architectural overview of IoT-SecureGate as one system with two runtimes: the monitoring application on the operator PC and the gateway on the edge device. The figures show how the parts relate, which planes carry control and data, how an inbound message moves through the gateway, and where ports and trust material sit. Later chapters expand the monitoring application (Chapter 5), the gateway (Chapter 6), and encryption and decryption (Chapter 7). The design realises the requirements of Chapter 3 inside the scope and threat boundaries stated there.

4.1 System Top-Level Architecture

Figure 4.1 shows the whole platform at a glance. Edge sensors (or laboratory fake sensors) publish into the gateway over MQTTS. The gateway agent admits and processes that traffic together with Mosquitto and Kafka, then returns metrics and logs to the operator PC. The monitoring application on that PC provisions and operates the gateway over SSH and presents operational views from a local observability stack (Loki, Prometheus, and Grafana).

The split matches the operational context of Chapter 3: one product, two runtimes, with the gateway as the controlled entry point for inbound plant telemetry.

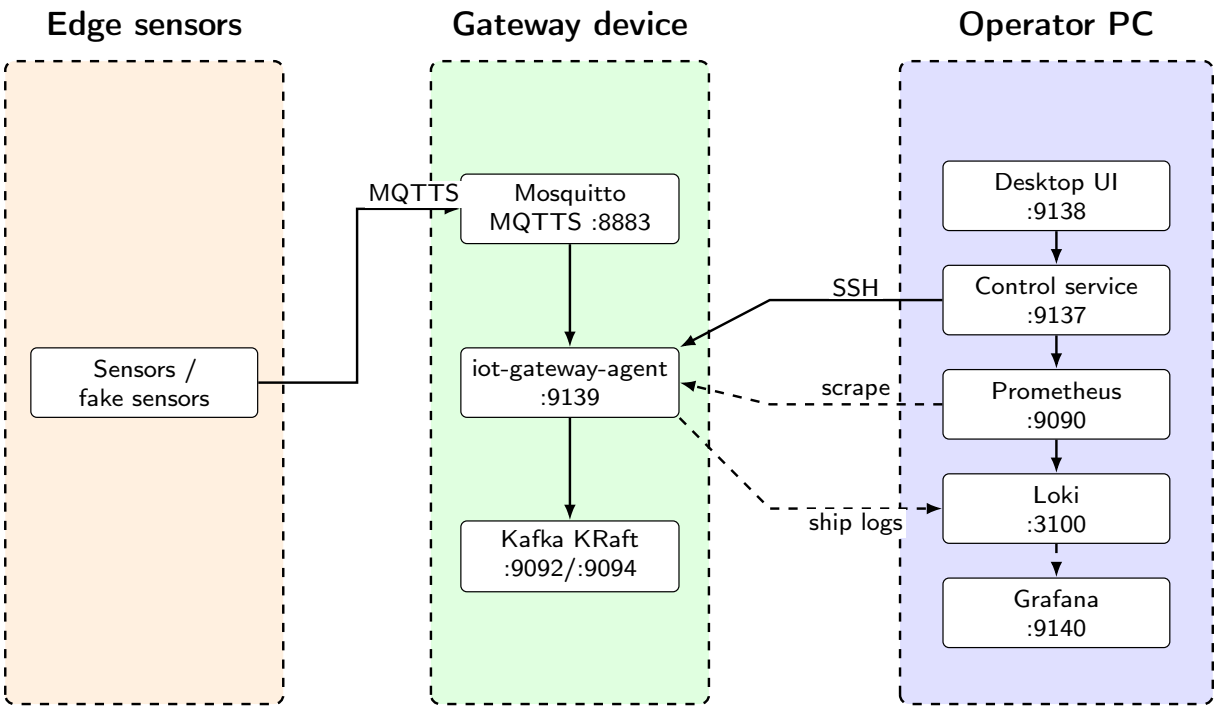


Figure 4.1: Top-level architecture of IoT-SecureGate: monitoring application, gateway, and sensors.

4.2 Monitoring Application Overview

Figure 4.2 summarises the operator runtime as two regions on the PC. The desktop package holds the Wizard/UI, control service, local registry, OS keyring, and the FakeCertRenewer used for laboratory fakes. Beside it, a local observability stack runs Loki, Prometheus, and Grafana as native processes under the application data directory so routine use does not depend on Docker (FR-13, NFR-1, NFR-2). The control service queries Loki and Prometheus; the UI opens Grafana in the system browser. Sensitive material for CA signing and day-2 privilege lives in the OS keyring (FR-14, NFR-6).

Chapter 5 details how this runtime is prepared, how it installs gateways and onboards devices, and how its operational workflows use the local stack.

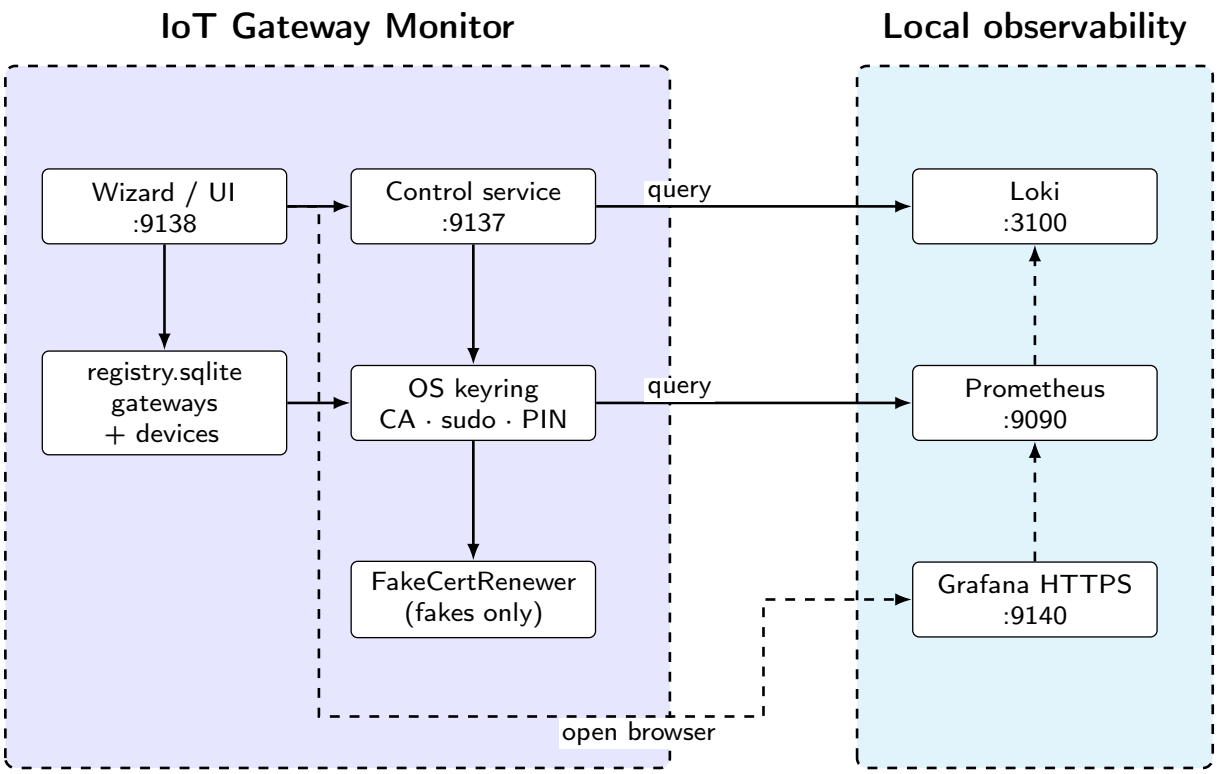


Figure 4.2: Overview of the monitoring application on the operator PC.

4.3 Gateway Overview

Figure 4.3 summarises the edge runtime from `architecture.md`. Real sensors and laboratory fakes share one MQTTS ingress into Mosquitto (fakes reach the broker through the on-gateway `iotgw-fake` bridge). The decrypt worker opens accepted envelopes, writes `sensor_data.log`, and produces Kafka topics. The Loki shipper pushes log lines to the operator PC; Prometheus scrapes agent metrics on port 9139.

Operational health views use the canonical application order Agent → MQTT → Decrypt → Shipper → Kafka. Chapter 6 expands how the agent and broker handle inbound traffic; Chapter 7 expands how envelopes are opened and validated.

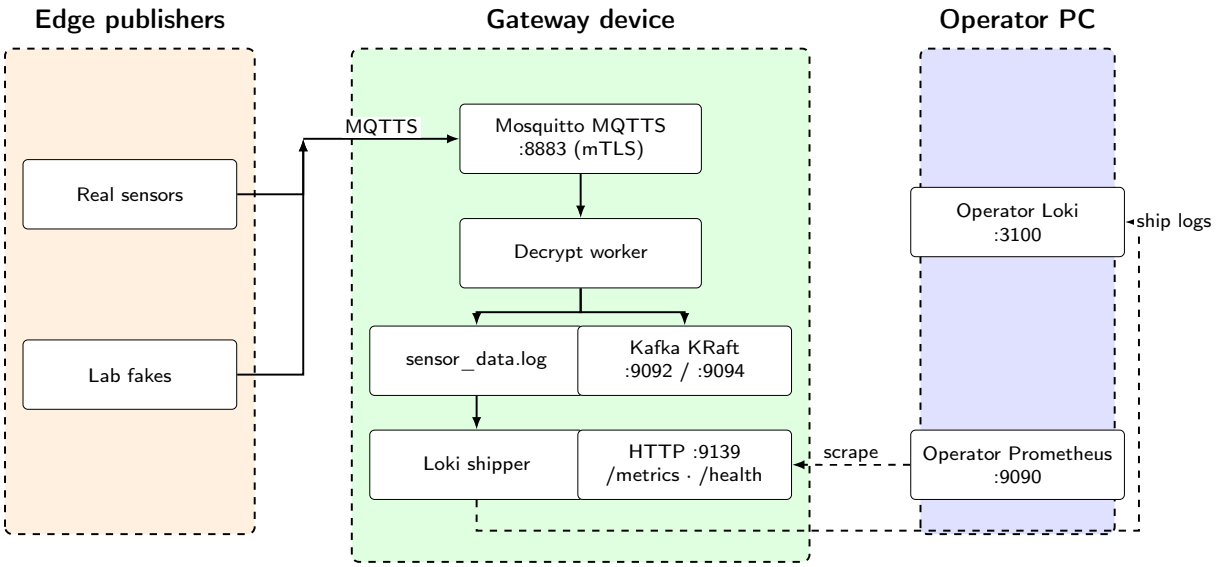


Figure 4.3: Overview of the gateway runtime and its links to operator observability.

4.4 Control Plane and Data Plane

IoT-SecureGate separates *control* actions from *data* movement, as shown in Figure 4.4.

- **Control plane.** The monitoring application reaches the gateway over SSH to provision the agent stack and to apply day-2 changes such as device registration, ACL updates, certificate work, and Auto prevent policy. Registry state and operator secrets stay on the PC.
- **Data plane.** Sensors publish on MQTTS into Mosquitto. After decrypt and schema validation, accepted plaintext is stored locally and in Kafka. Metrics are scraped over HTTP from the agent; logs are pushed to operator Loki.

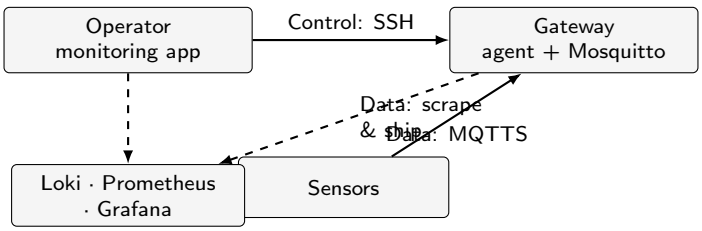


Figure 4.4: Control plane (SSH) and data plane (MQTTS and HTTP observability) between operator PC, gateway, and sensors.

That separation keeps privileged configuration off the sensor path and keeps high-volume telemetry off the SSH session used for administration.

4.5 End-to-End Inbound Path

Figure 4.5 is the inbound path that FR-1 through FR-4 require. A device may publish only after registration has placed MQTT credentials, ACL entries, a client certificate, and

a Kafka topic on the gateway. The broker enforces authentication and authorization. The decrypt worker opens the application envelope (or legacy Fernet where still accepted), validates the schema, then writes plaintext to `sensor_data.log` and to a per-device Kafka topic `sensor.<device_id>`. The shipper tails the log toward operator Loki; Prometheus scrapes agent metrics independently.

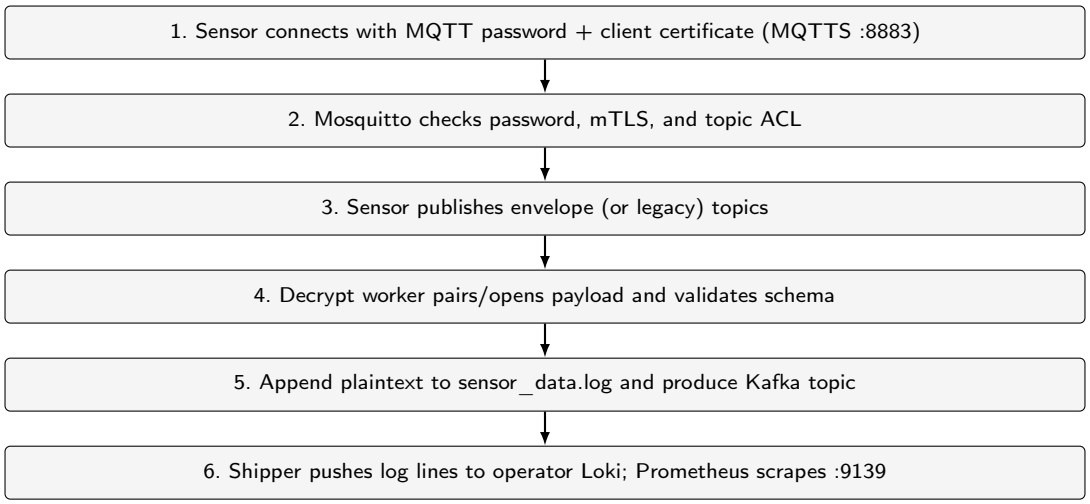


Figure 4.5: End-to-end inbound path from sensor publish to storage and operator observability.

Failed authentication, authorization, decrypt, or schema checks do not enter the accepted storage path; they feed deny and invalid signalling used for operator visibility and response (FR-9–FR-11). Chapter 6 describes this path as gateway behaviour; Chapter 7 describes the envelope open in detail.

4.6 Layered Message Protection Overview

Inbound protection uses independent layers, summarised in Figure 4.6 and Table 4.1. Transport security answers who may connect and publish. Application-layer encryption answers confidentiality of the JSON payload after TLS terminates on the gateway. Schema validation answers whether the recovered message has the expected shape.

Layer	**Mechanism**	**Answers**
Transport	MQTTS with mutual TLS, Mosquitto password, and topic ACL	Who may connect and publish
Application	Envelope v1 (`v1-psk` / `v1-ecdhe`)	Confidentiality after TLS termination
Schema	Registered JSON schema on the gateway	Message shape after decrypt
Day material	Daily Message Encryption Certificate (DMEC)	Day-scoped wrap for `v1-ecdhe`

Table 4.1: Protection layers on the gateway inbound path (overview).

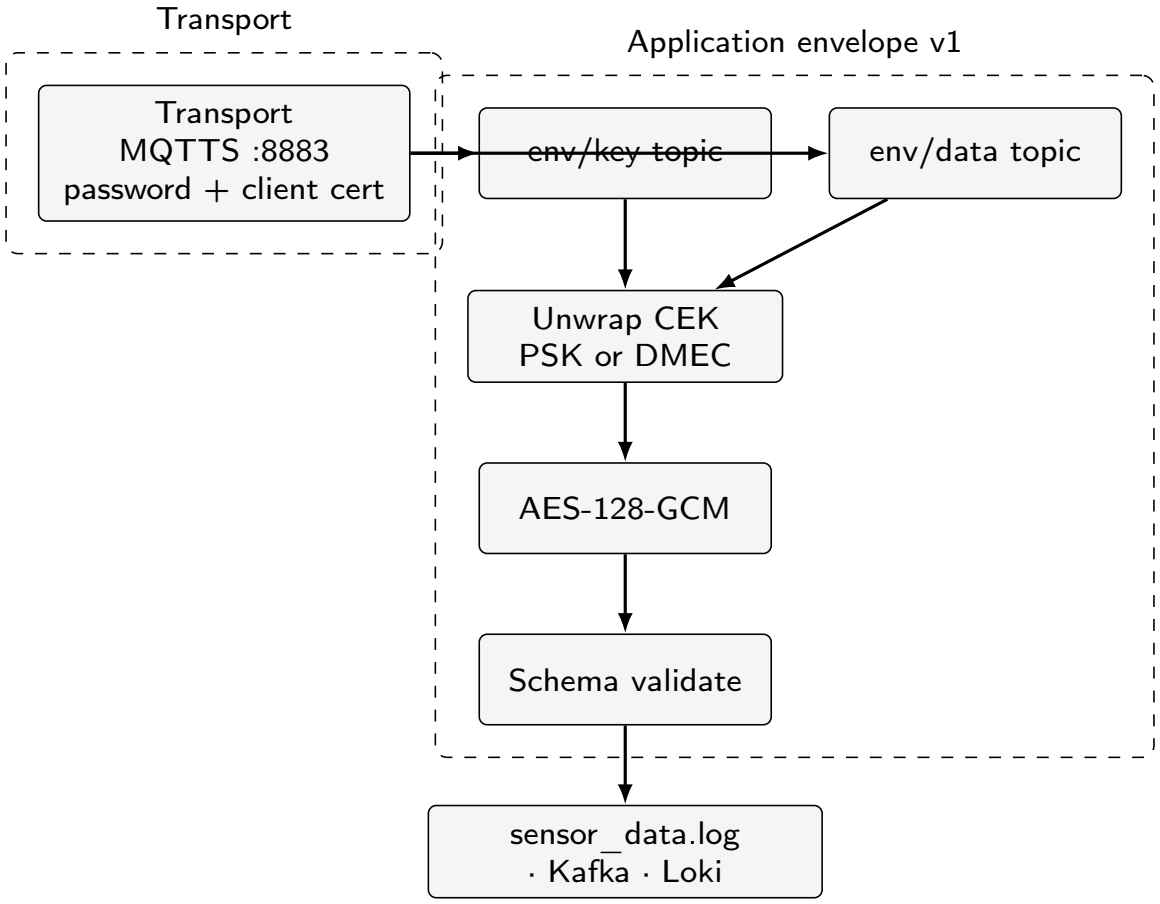


Figure 4.6: Layered protection on the inbound path: MQTTS transport and application envelope v1.

Profiles, topic pairing, replay resistance, and the full encrypt/decrypt workflow are specified in Chapter 7. Legacy Fernet remains accepted only where still configured; the architectural target is envelope v1 on MQTTS.

4.7 Deployment Topology and Ports

Table 4.2 and Table 4.3 list the default binds for a trusted plant or laboratory LAN (NFR-9). The operator stack listens on the PC; each gateway exposes MQTTS, agent HTTP, and Kafka. Figure 4.7 places those ports on the same top-level split as Figure 4.1.

Component	**Bind**	**Role**
Control service	`127.0.0.1:9137`	FastAPI registry, playbooks, queries
Wizard / UI	`127.0.0.1:9138`	Desktop interface and API proxy
Loki	`:3100`	Log target for gateway shipper
Prometheus	`:9090`	Scrapes gateway agent metrics
Grafana	`127.0.0.1:9140` (HTTPS)	Local dashboards in system browser

Table 4.2: Operator PC ports for the monitoring application.

Component	**Bind**	**Role**
Agent HTTP	`:9139`	Metrics, health, day-crypto helpers
Mosquitto MQTTS	`:8883`	Sensor ingress (password + client cert)
Kafka INTERNAL	`127.0.0.1:9092`	Agent produce path
Kafka EXTERNAL	gateway IP `:9094`	Trusted-LAN Grafana datasource

Table 4.3: Gateway device ports.

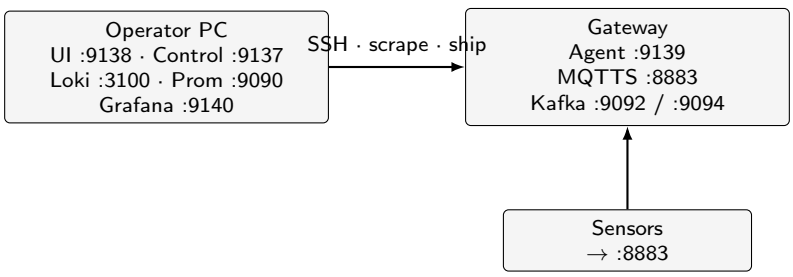


Figure 4.7: Deployment topology and principal ports on a trusted LAN.

4.8 Trust Material and Secrets

Figure 4.8 shows where trust material lives. The gateway holds its CA, server certificate, device client certificates, ACL and password files, and per-device crypto material under the agent data directory. The operator PC stores gateway and device records in `registry.sqlite` and keeps CA passphrases, gateway sudo credentials, and the local admin PIN in the OS keyring (with a documented file fallback). Provision passes privileged passwords on SSH standard input rather than on remote argument lists (NFR-6). The local proxy injects keyring secrets into control-service requests when the operator does not re-enter them for every day-2 action.

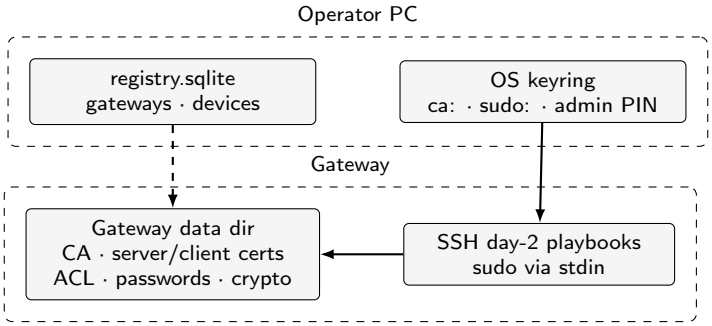


Figure 4.8: Overview of trust material on the operator PC and on the gateway.

Ordinary settings files and application logs are not the store for those secrets. Certificate lifecycle and related workflows are detailed with the runtimes that implement them.

4.9 Relation to Following Chapters

The figures in this chapter are the map of the system. The next chapters zoom into each concern without repeating the same overview diagrams:

- Chapter 5 details the monitoring application: architecture, interaction with the gateway for installation and device onboarding, and operational workflows—building on Figures 4.2, 4.4, and 4.8.
- Chapter 6 details the gateway: agent and MQTT handling, the inbound processing path, APIs and control interfaces, and detection and response—building on Figures 4.3 and 4.5.
- Chapter 7 details transport and application-layer protection, the encrypt/decrypt workflow, and schema validation after decrypt—building on Figure 4.6.

Chapter 5

Monitoring Application

5.1 Architecture of the Monitoring Application

5.2 Interaction with the Gateway

5.2.1 Gateway discovery and installation

5.2.2 Sensor and fake-sensor onboarding

5.3 Functions and Operational Workflows

5.3.1 Local monitoring preparation

5.3.2 Observing metrics and transmission activity

5.3.3 Control and laboratory exercise

5.3.4 Lifecycle of the operator installation

Chapter 6

Gateway

6.1 Architecture of the Gateway

6.2 Agent Operation and MQTT Handling

6.3 Inbound Processing Path

6.4 Gateway APIs and Control Interfaces

6.5 Detection and Response Behaviour

Chapter 7

Encryption and Decryption

7.1 Protection Model

7.2 Transport Security and Mutual TLS

7.3 Application-Layer Envelope

7.4 Encryption and Decryption Workflow

7.5 Schema Validation and Accepted Storage

Chapter 8

Evaluation

8.1 Evaluation Goals and Method

8.2 Admission and Authentication

8.3 Protected Transmission and Storage

8.4 Monitoring and Detection

8.5 Resource Use and Limitations

Chapter 9

Conclusion and Future Work

9.1 Summary

9.2 Contributions

9.3 Future Work

Bibliography

[1] OASIS, *MQTT Version 5.0*, OASIS Standard, 7 March 2019. https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html

[2] Eclipse Mosquitto, *mosquitto.conf Man Page*. https://mosquitto.org/man/mosquitto-conf-5.html

[3] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, 2018. https://www.rfc-editor.org/rfc/rfc8446

[4] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*, RFC 5280, 2008. https://www.rfc-editor.org/rfc/rfc5280

[5] Prometheus Authors, *Overview*. https://prometheus.io/docs/introduction/overview/

[6] Grafana Labs, *Loki Overview*. https://grafana.com/docs/loki/latest/get-started/overview/

[7] C. Sengul and A. Kirby, *Message Queuing Telemetry Transport (MQTT) and Transport Layer Security (TLS) Profile of Authentication and Authorization for Constrained Environments (ACE) Framework*, RFC 9431, 2023. https://www.rfc-editor.org/rfc/rfc9431

[8] I. Vaccari, G. Chiola, M. Aiello, M. Mongelli, and E. Cambiaso, "MQTTset, a New Dataset for Machine Learning Techniques on MQTT," *Sensors*, vol. 20, no. 22, article 6578, 2020. https://doi.org/10.3390/s20226578

[9] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, D. Breitenbacher, A. Shabtai, and Y. Elovici, "N-BaIoT: Network-based Detection of IoT Botnet Attacks Using Deep Autoencoders," arXiv:1805.03409, 2018. https://arxiv.org/abs/1805.03409

[10] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *Advances in Neural Information Processing Systems*, vol. 30, 2017. https://proceedings.neurips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree.pdf